

ttydiff — Problem Statement

Track: B — Parsers & Data Formats **Team:** Siddhesh, Sarvesh, Shubham (Lead) **Rule:** Standard library only. Zero third-party dependencies.

The Problem

Terminal programs (like `vim`, `htop`, `ls --color`) don't just print plain text — they send hidden formatting codes too, called **ANSI escape sequences**. These codes say things like "move cursor here," "make this red," "clear the screen."

Right now, there's no simple, dependency-free way to:

Understand what a terminal screen actually looks like at any moment

Know exactly **what changed** between two moments, instead of re-reading everything

This matters for things like screen readers (for visually impaired users), terminal monitoring tools, and session recording/replay tools.

What We're Building

ttydiff — a tool that:

Reads raw terminal output (including the hidden escape codes)

Builds a virtual model of what's on screen

Compares two moments in time and reports **only what changed**

Can run multiple commands one after another and show a diff after each

Example usage:

```
 ttydiff run "ls -la" "htop" "git status"
```

Each command runs, ttydiff watches the screen, and prints what changed.

Why This Fits the Hackathon

100% buildable with Python's standard library — no installs needed

Real, hard parsing problem (exactly what Track B wants: handle ugly edge cases, not just the easy stuff)

Genuinely useful — this is the missing piece behind real accessibility tools

Strong demo: watch real commands run live with real diffs printed

The Pipeline (Simple View)



Team Split (Short Version)

Person	Job
Siddhesh	Decode the hidden terminal codes + build the virtual screen
Sarvesh	Compare screens to find changes + run commands one after another
Shubham	Tie it all together, build the CLI, write docs, handle testing & demo

What We're NOT Building

- No text-to-speech / actual voice output (that's a future layer, not this weekend)
- No support for every possible terminal feature — we pick the common ones and clearly list what we skip
- No Windows support (the pty trick we use is Unix-only) — Mac/Linux only

Timeline

- Day 1:** Build each piece separately (parser, screen, runner)
- Day 2:** Connect everything, get one real command working end-to-end
- Day 3:** Polish, test, write docs, record demo, submit

Full detailed plan with hour-by-hour breakdown: see [ttydiff-plan.md](#)